AULA 13 — MEU PRIMEIRO SISTEMA COM BANCO DE DADOS

UC5 — Front-end de Projetos Web | 4 horas

Projeto: StockTech — Controle de Estoque

O que vamos construir?

Um pequeno sistema para uma loja de tecnologia que precisa controlar seus produtos.

Ao final da aula, o StockTech deverá permitir:

- Cadastrar produtos com nome, categoria, preço e quantidade.
- Consultar os produtos cadastrados.
- Editar informações de um produto.
- Excluir produtos com confirmação.
- Calcular o valor total do estoque.
- Guardar as informações em um banco SQLite, e não no localStorage.

1 — O que muda nesta aula?

Até agora:

HTML + CSS + JavaScript

↓

localStorage

Agora:

FRONT-END

HTML + CSS + JavaScript

BACK-END

Python recebe e processa as solicitações

BANCO DE DADOS

SQLite armazena os produtos

HTML monta a tela

CSS organiza a aparência

JavaScript envia as solicitações

Python conversa com o banco

SQLite guarda os registros.

Comandos e conceitos novos

Termo	**O que significa**
Back-end	Parte do sistema que processa solicitações e acessa os dados
SQLite	Banco de dados que utiliza um arquivo local
API	Meio pelo qual o front-end se comunica com o back-end
fetch()	Comando JavaScript para fazer solicitações à API
SQL	Linguagem utilizada para trabalhar com dados no banco
SELECT	Consultar
INSERT	Cadastrar
UPDATE	Atualizar
DELETE	Excluir

2 — Preparando o projeto no VS Code

Crie uma pasta chamada:

Aula13_StockTech

Dentro dela, crie os quatro arquivos:

Aula13_StockTech

│

├── index.html

├── style.css

├── script.js

└── servidor.py

O arquivo estoque.db será criado automaticamente pelo Python. Não é necessário criá-lo manualmente.

Teste inicial do Python

No VS Code, abra Terminal → New Terminal.

Digite:

python --version

Se não funcionar, teste:

py --version

Deverá aparecer algo parecido com Python 3.x.x. O número da versão pode variar.

Importante: para esta aula, usaremos o terminal integrado do VS Code. Não precisamos instalar Flask, MySQL ou extensões adicionais.

3 — CONSTRUINDO O HTML

Abra o arquivo index.html.

BLOCO 1 — Estrutura, cabeçalho e menu

Comandos: header cria o cabeçalho; nav organiza a navegação; main identifica o conteúdo principal.

COPIE E COLE

```
<!DOCTYPE html>

<html lang="pt-BR">


<head>
    <!-- CONFIGURAÇÕES -->
    <meta charset="UTF-8">
    <meta name="viewport"
        content="width=device-width, initial-scale=1.0">
```

```
    <title>StockTech - Controle de Estoque</title>

    <!-- CONECTA AO CSS -->
    <link rel="stylesheet" href="style.css">
</head>

<body>

    <!-- CABEÇALHO -->
    <header>
        <h1>📦 StockTech</h1>
        <p>Sistema de Controle de Estoque</p>
    </header>

    <!-- MENU -->
    <nav>
        <a href="#cadastro">Cadastrar</a>
        <a href="#estoque">Consultar Estoque</a>
    </nav>
```

O menu utiliza links internos com #, que levam até as áreas identificadas pelo mesmo id.

BLOCO 2 — Formulário de produtos

Vamos criar o formulário completo, com todos os campos e o botão.

Comandos: type="number" recebe valores numéricos; step="0.01" permite centavos; onsubmit executa uma função quando o formulário é enviado.

COPIE E COLE LOGO DEPOIS DE </nav>, ANTES DE </body>

```
<!-- CONTEÚDO PRINCIPAL -->
<main>

  <!-- ÁREA DE CADASTRO -->
  <section id="cadastro">

    <h2>➕ Cadastro de Produtos</h2>

    <!-- FORMULÁRIO COMPLETO -->
    <form id="formProduto"
        onsubmit="salvarProduto(event)">

      <label for="nome">Nome do produto:</label>
      <input type="text"
          id="nome"
          placeholder="Ex.: Mouse Gamer"
          required>

      <label for="categoria">Categoria:</label>
      <select id="categoria" required>
        <option value="">Selecione</option>
        <option>Periféricos</option>
        <option>Computadores</option>
        <option>Celulares</option>
        <option>Acessórios</option>
        <option>Outros</option>
```

```
</select>

<label for="preco">Preço (R$):</label>
<input type="number"
       id="preco"
       min="0.01"
       step="0.01"
       required>

<label for="quantidade">Quantidade:</label>
<input type="number"
       id="quantidade"
       min="0"
       step="1"
       required>

<!-- BOTÃO DE CADASTRO OU EDIÇÃO -->
<button type="submit" id="botaoSalvar">
   💾 Salvar Produto
</button>

<button type="button"
        class="secundario"
        onclick="limparFormulario()">
    Limpar
</button>
```

```
    </form>

  </section>

</main>
```

BLOCO 3 — Consulta e indicadores

Agora vamos criar a área que receberá os produtos vindos do banco.

Comandos: id identifica os elementos que serão atualizados pelo JavaScript; tbody organiza as linhas da tabela.

COPIE E COLE DEPOIS DE </main>, ANTES DE </body>

```
<!-- ÁREA DE CONSULTA -->
<section id="estoque" class="area-estoque">

  <h2>📋 Produtos Cadastrados</h2>

  <!-- INDICADORES -->
  <div class="indicadores">

    <div class="indicador">
      <h3>Produtos</h3>
      <p id="totalProdutos">0</p>
    </div>

    <div class="indicador">
      <h3>Unidades</h3>
```

```
        <p id="totalUnidades">0</p>
    </div>

    <div class="indicador">
        <h3>Valor do Estoque</h3>
        <p id="valorEstoque">R$ 0,00</p>
    </div>

</div>

<!-- TABELA DE PRODUTOS -->
<div class="tabela-container">

    <table>
        <thead>
            <tr>
                <th>Produto</th>
                <th>Categoria</th>
                <th>Preço</th>
                <th>Qtd.</th>
                <th>Ações</th>
            </tr>
        </thead>

        <tbody id="listaProdutos">
            <!-- JAVASCRIPT INSERE OS PRODUTOS -->
        </tbody>
```

```
        </table>

    </div>

</section>

<!-- RODAPÉ -->
<footer>
    <p>StockTech — Projeto Educacional</p>
</footer>

<!-- CONECTA AO JAVASCRIPT -->
<script src="script.js"></script>
</body>
</html>
```

Salve o arquivo.

Ainda não vamos abrir pelo Live Server, porque o projeto terá um servidor Python próprio.

4 — ESTILIZANDO O SISTEMA

Abra style.css.

BLOCO 1 — Aparência geral

Comandos: box-sizing controla o tamanho dos elementos; max-width limita a largura; margin: auto ajuda a centralizar.

COPIE E COLE

```
/* CONFIGURAÇÕES GERAIS */
* {
```

```
    box-sizing: border-box;
}

body {
    margin: 0;
    font-family: Arial, sans-serif;
    background-color: #f1f5f9;
    color: #1e293b;
}

/* CABEÇALHO */
header {
    background-color: #0f172a;
    color: white;
    text-align: center;
    padding: 30px;
}

header h1 {
    margin: 0;
    color: #38bdf8;
}

/* MENU */
nav {
    display: flex;
    justify-content: center;
```

```
    gap: 20px;
    background-color: #1e293b;
    padding: 15px;
}

nav a {
    color: white;
    text-decoration: none;
    font-weight: bold;
}

nav a:hover {
    color: #38bdf8;
}

/* ÁREAS PRINCIPAIS */
main,
.area-estoque {
    width: 90%;
    max-width: 1100px;
    margin: 30px auto;
}

section {
    background-color: white;
    padding: 25px;
    border-radius: 12px;
```

```
}
```

BLOCO 2 — Formulário e botões

Comandos: display: flex organiza elementos; flex-direction: column coloca os campos um abaixo do outro; hover altera a aparência ao passar o mouse.

COPIE E COLE ABAIXO

```
/* FORMULÁRIO */
form {
    display: flex;
    flex-direction: column;
    gap: 12px;
}

input,
select {
    padding: 12px;
    border: 1px solid #cbd5e1;
    border-radius: 8px;
    font-size: 16px;
}

/* BOTÕES */
button {
    background-color: #0284c7;
    color: white;
    border: none;
    padding: 12px;
```

```
    border-radius: 8px;
    cursor: pointer;
}

button:hover {
    background-color: #0369a1;
}

.secundario {
    background-color: #64748b;
}

.secundario:hover {
    background-color: #475569;
}

.excluir {
    background-color: #dc2626;
}

.excluir:hover {
    background-color: #b91c1c;
}
```

BLOCO 3 — Indicadores e tabela

Comandos: grid organiza os indicadores em colunas; overflow-x: auto permite rolar a tabela horizontalmente em telas pequenas.

COPIE E COLE ABAIXO

```
/* INDICADORES */
.indicadores {
    display: grid;
    grid-template-columns: repeat(3, 1fr);
    gap: 15px;
    margin-bottom: 25px;
}

.indicador {
    background-color: #e0f2fe;
    padding: 20px;
    border-radius: 10px;
    text-align: center;
}

.indicador p {
    font-size: 24px;
    font-weight: bold;
    color: #0369a1;
}

/* TABELA */
.tabela-container {
    overflow-x: auto;
}
```

```
table {
    width: 100%;
    border-collapse: collapse;
}

th,
td {
    padding: 12px;
    border-bottom: 1px solid #cbd5e1;
    text-align: left;
}

th {
    background-color: #e2e8f0;
}

td button {
    margin: 3px;
}

/* RODAPÉ */
footer {
    text-align: center;
    padding: 20px;
    color: #64748b;
}
```

```
/* RESPONSIVIDADE */
@media (max-width: 600px) {

  .indicadores {
    grid-template-columns: 1fr;
  }

  nav {
    flex-direction: column;
    align-items: center;
  }

}
```

Pronto! Todo o CSS está concluído.

5 — CRIANDO O BANCO DE DADOS REAL

Abra servidor.py.

Vamos utilizar o SQLite, que acompanha o Python. Ele criará automaticamente o arquivo estoque.db.

BLOCO 1 — Importações e criação da tabela

Comandos novos:

- import → utiliza recursos do Python.
- sqlite3.connect() → conecta ao banco de dados.
- CREATE TABLE IF NOT EXISTS → cria a tabela somente se ela ainda não existir.
- PRIMARY KEY → identifica cada produto.
- commit() → confirma as alterações.

COPIE E COLE

```
# RECURSOS DO PYTHON
import json
import sqlite3
from http.server import HTTPServer, SimpleHTTPRequestHandler
from urllib.parse import urlparse


# NOME DO ARQUIVO DO BANCO
BANCO = "estoque.db"


# CRIA A TABELA DE PRODUTOS
def criar_banco():

    conexao = sqlite3.connect(BANCO)

    conexao.execute("""
        CREATE TABLE IF NOT EXISTS produtos (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            nome TEXT NOT NULL,
            categoria TEXT NOT NULL,
            preco REAL NOT NULL,
            quantidade INTEGER NOT NULL
        )
    """)

    conexao.commit()
```

```
conexao.close()
```

Estamos criando uma tabela chamada produtos, com cinco colunas. O id será gerado automaticamente para cada produto.

BLOCO 2 — Preparando o servidor

O servidor será responsável por receber solicitações do JavaScript.

Comandos novos:

- class → organiza as funcionalidades do servidor.
- do_GET → responde a solicitações de consulta.
- do_POST → responde a solicitações de cadastro.
- json.dumps() → transforma dados Python em texto JSON.

COPIE E COLE ABAIXO

```
# SERVIDOR QUE ATENDE AS SOLICITAÇÕES DO SITE
class Servidor(SimpleHTTPRequestHandler):

    # ENVIA RESPOSTAS EM JSON
    def responder(self, dados, status=200):

        conteudo = json.dumps(
            dados,
            ensure_ascii=False
        ).encode("utf-8")

        self.send_response(status)
        self.send_header(
```

```
        "Content-Type",
        "application/json; charset=utf-8"
    )
    self.send_header(
        "Content-Length",
        str(len(conteudo))
    )
    self.end_headers()

    self.wfile.write(conteudo)


# LÊ OS DADOS ENVIADOS PELO JAVASCRIPT
def ler_dados(self):

    tamanho = int(
        self.headers.get("Content-Length", 0)
    )

    return json.loads(
        self.rfile.read(tamanho)
    )


# CONSULTA OS PRODUTOS
def do_GET(self):
```

```
        caminho = urlparse(self.path).path

        if caminho == "/api/produtos":

            conexao = sqlite3.connect(BANCO)
            conexao.row_factory = sqlite3.Row

            produtos = conexao.execute(
                "SELECT * FROM produtos ORDER BY id DESC"
            ).fetchall()

            conexao.close()

            self.responder([
                dict(produto) for produto in produtos
            ])

        else:

            # ABRE NORMALMENTE HTML, CSS E JAVASCRIPT
            super().do_GET()
```

O que acontece aqui?

Quando o JavaScript pedir:

/api/produtos

o Python consultará o banco e devolverá a lista dos produtos.

Importante: esta API é local e didática. Não estamos publicando o sistema na internet.

BLOCO 3 — Cadastrar produtos

Comandos novos:

- INSERT INTO → insere um registro.
- ? → recebe valores separadamente do comando SQL, evitando montar SQL com texto digitado pelo usuário.
- POST → tipo de solicitação utilizada para cadastrar.

COPIE E COLE ABAIXO DO BLOCO ANTERIOR

Este é um método da classe Servidor. Mantenha a indentação de quatro espaços antes de def, como no bloco anterior.

```
    # CADASTRA UM PRODUTO
    def do_POST(self):

        if self.path != "/api/produtos":
            self.responder({"erro": "Rota inválida"}, 404)
            return

        dados = self.ler_dados()

        # VALIDA OS VALORES RECEBIDOS
        try:
            nome = str(dados["nome"]).strip()
            categoria = str(dados["categoria"]).strip()
            preco = float(dados["preco"])
            quantidade = int(dados["quantidade"])

            if (
                not nome or
                not categoria or
```

```
            preco <= 0 or
            quantidade < 0
        ):
            raise ValueError()

    except (KeyError, TypeError, ValueError):
        self.responder({"erro": "Dados inválidos"}, 400)
        return

    conexao = sqlite3.connect(BANCO)

    conexao.execute(
        """INSERT INTO produtos
           (nome, categoria, preco, quantidade)
           VALUES (?, ?, ?, ?)""",
        (nome, categoria, preco, quantidade)
    )

    conexao.commit()
    conexao.close()

    self.responder(
        {"mensagem": "Produto cadastrado!"},
        201
    )
```

BLOCO 4 — Editar produtos

Comando novo: UPDATE altera um registro existente. O WHERE id = ? identifica qual produto deve ser alterado.

COPIE E COLE ABAIXO

```
    # ATUALIZA UM PRODUTO
    def do_PUT(self):

        try:
            id_produto = int(self.path.split("/")[-1])
            dados = self.ler_dados()

            nome = str(dados["nome"]).strip()
            categoria = str(dados["categoria"]).strip()
            preco = float(dados["preco"])
            quantidade = int(dados["quantidade"])

            if (
                not nome or
                not categoria or
                preco <= 0 or
                quantidade < 0
            ):
                raise ValueError()

        except (ValueError, TypeError, KeyError):
            self.responder({"erro": "Dados inválidos"}, 400)
            return

        conexao = sqlite3.connect(BANCO)
```

```
        cursor = conexao.execute(
            """UPDATE produtos
               SET nome = ?, categoria = ?,
                   preco = ?, quantidade = ?
               WHERE id = ?""",
            (nome, categoria, preco, quantidade, id_produto)
        )

        conexao.commit()
        alterados = cursor.rowcount
        conexao.close()

        if alterados == 0:
            self.responder({"erro": "Produto não encontrado"}, 404)
        else:
            self.responder({"mensagem": "Produto atualizado!"})
```

BLOCO 5 — Excluir produtos

Comando novo: DELETE FROM remove um registro. O WHERE garante que apenas o produto selecionado seja excluído.

COPIE E COLE ABAIXO

```
    # EXCLUI UM PRODUTO
    def do_DELETE(self):

        try:
            id_produto = int(self.path.split("/")[-1])
        except ValueError:
```

```
            self.responder({"erro": "ID inválido"}, 400)
            return

        conexao = sqlite3.connect(BANCO)

        cursor = conexao.execute(
            "DELETE FROM produtos WHERE id = ?",
            (id_produto,)
        )

        conexao.commit()
        excluidos = cursor.rowcount
        conexao.close()

        if excluidos == 0:
            self.responder({"erro": "Produto não encontrado"}, 404)
        else:
            self.responder({"mensagem": "Produto excluído!"})
```

BLOCO 6 — Iniciar o servidor

Agora precisamos de um comando para iniciar o sistema.

Comandos novos:

- HTTPServer → inicia o servidor.
- 127.0.0.1 → representa o próprio computador.
- 8000 → número da porta utilizada.

- serve_forever() → mantém o servidor funcionando até ser interrompido.

COPIE E COLE ABAIXO

Este bloco começa sem espaços antes de if, pois fica fora da classe.

```
# INICIA O BANCO E O SERVIDOR
if __name__ == "__main__":

    criar_banco()

    servidor = HTTPServer(
        ("127.0.0.1", 8000),
        Servidor
    )

    print("StockTech funcionando!")
    print("Acesse: http://127.0.0.1:8000")

    servidor.serve_forever()
```

TESTE INICIAL DO SERVIDOR (ANTES DO JAVASCRIPT)

Salve todos os arquivos.

No terminal integrado do VS Code, dentro da pasta Aula13_StockTech, execute:

python servidor.py

Se o computador utilizar o comando py, execute:

py servidor.py

Deverá aparecer:

StockTech funcionando!

Acesse: http://127.0.0.1:8000

Abra no navegador:

http://127.0.0.1:8000

A página deverá aparecer com o layout que construímos.

Primeira conquista: servidor funcionando e arquivo do banco criado. O cadastro será testado depois de concluir o JavaScript.

O arquivo estoque.db deverá aparecer na pasta do projeto. Não precisa abri-lo nem editar seu conteúdo.

Atenção: deixe o terminal aberto. Se encerrar o servidor, o sistema deixará de responder às solicitações do JavaScript.

6 — CONECTANDO O JAVASCRIPT AO BANCO

Agora vamos fazer o front-end conversar com o servidor Python.

Abra script.js.

BLOCO 1 — Variáveis e consulta ao banco

Comandos novos:

- fetch() → faz uma solicitação ao servidor.
- await → espera a resposta chegar.
- async → permite utilizar await na função.
- .json() → transforma a resposta JSON em dados JavaScript.

COPIE E COLE

```
// GUARDA A LISTA RECEBIDA DO BANCO

let produtos = [];


// IDENTIFICA SE ESTAMOS EDITANDO

let idEdicao = null;



// CONSULTA OS PRODUTOS NO SERVIDOR
```

```
async function carregarProdutos() {

    try {

        // PEDE OS PRODUTOS AO PYTHON
        let resposta = await fetch("/api/produtos");

        if (!resposta.ok) {
            throw new Error("Falha na consulta");
        }

        // TRANSFORMA A RESPOSTA EM ARRAY
        produtos = await resposta.json();

        // ATUALIZA A TELA
        mostrarProdutos();

    } catch (erro) {

        alert("Não foi possível consultar o banco.");

    }

}
```

BLOCO 2 — Cadastrar e atualizar

Esta função recebe os dados do formulário e envia ao Python.

Comandos novos:

- event.preventDefault() → impede o recarregamento automático do formulário.
- method: "POST" → cadastra.
- method: "PUT" → atualiza.
- JSON.stringify() → transforma o objeto em texto JSON.

COPIE E COLE ABAIXO

```
// CADASTRA OU ATUALIZA UM PRODUTO
async function salvarProduto(event) {

    event.preventDefault();

    // PEGA OS DADOS DO FORMULÁRIO
    let produto = {

        nome: document.getElementById("nome").value.trim(),

        categoria: document.getElementById("categoria").value,

        preco: Number(
            document.getElementById("preco").value
        ),

        quantidade: Number(
            document.getElementById("quantidade").value
        )

    };
```

```
// VALIDA OS DADOS
if (
    produto.nome == "" ||
    produto.categoria == "" ||
    produto.preco <= 0 ||
    !Number.isInteger(produto.quantidade) ||
    produto.quantidade < 0
) {
    alert("Preencha os campos corretamente.");
    return;
}

// DEFINE SE É CADASTRO OU EDIÇÃO
let endereco = "/api/produtos";
let metodo = "POST";

if (idEdicao != null) {
    endereco += "/" + idEdicao;
    metodo = "PUT";
}

try {

    // ENVIA OS DADOS PARA O PYTHON
    let resposta = await fetch(endereco, {
```

```
            method: metodo,

            headers: {
                "Content-Type": "application/json"
            },

            body: JSON.stringify(produto)

        });

        if (!resposta.ok) {
            throw new Error("Falha ao salvar");
        }

        alert("Produto salvo com sucesso!");

        limparFormulario();
        await carregarProdutos();

    } catch (erro) {

        alert("Não foi possível salvar o produto.");

    }

}
```

BLOCO 3 — Mostrar os produtos

Agora vamos apresentar os dados recebidos do SQLite.

Comandos: for percorre os produtos; createElement() cria elementos HTML; textContent coloca texto na página; toFixed(2) mostra duas casas decimais.

COPIE E COLE ABAIXO

```
// MOSTRA OS PRODUTOS NA TABELA
function mostrarProdutos() {

    let lista =
        document.getElementById("listaProdutos");

    lista.innerHTML = "";

    let unidades = 0;
    let valorTotal = 0;

    // PERCORRE OS PRODUTOS
    for (let produto of produtos) {

        let linha = document.createElement("tr");

        // CRIA AS COLUNAS COM OS DADOS
        let dados = [
            produto.nome,
            produto.categoria,
            "R$ " + produto.preco.toFixed(2),
            produto.quantidade
        ];
```

```
for (let dado of dados) {

    let coluna = document.createElement("td");
    coluna.textContent = dado;
    linha.appendChild(coluna);

}

// CRIA OS BOTÕES DE AÇÃO
let colunaAcoes = document.createElement("td");

let botaoEditar = document.createElement("button");
botaoEditar.textContent = "✏️ Editar";
botaoEditar.onclick = function () {
    editarProduto(produto.id);
};

let botaoExcluir = document.createElement("button");
botaoExcluir.textContent = "🗑️ Excluir";
botaoExcluir.className = "excluir";
botaoExcluir.onclick = function () {
    excluirProduto(produto.id);
};

colunaAcoes.appendChild(botaoEditar);
colunaAcoes.appendChild(botaoExcluir);
```

```
        linha.appendChild(colunaAcoes);

        lista.appendChild(linha);

        // CALCULA OS INDICADORES
        unidades += produto.quantidade;

        valorTotal +=
            produto.preco * produto.quantidade;

    }

    // ATUALIZA O RESUMO DO ESTOQUE
    document.getElementById("totalProdutos").textContent =
        produtos.length;

    document.getElementById("totalUnidades").textContent =
        unidades;

    document.getElementById("valorEstoque").textContent =
        valorTotal.toLocaleString("pt-BR", {
            style: "currency",
            currency: "BRL"
        });

}
```

TESTE DO CADASTRO

Atualize a página no navegador.

Cadastre:

Campo	**Informação**
Nome	Mouse Gamer
Categoria	Periféricos
Preço	120
Quantidade	5

Clique em Salvar Produto.

O produto deverá aparecer na tabela.

O indicador de valor do estoque deverá mostrar:

Valor do estoque

R$ 600,00

5 unidades × R$ 120,00

BLOCO 4 — Editar um produto

Comandos: find() localiza um objeto no Array; .value preenche os campos; scrollIntoView() leva a tela até o formulário.

COPIE E COLE ABAIXO

```
// PREPARA UM PRODUTO PARA EDIÇÃO

function editarProduto(id) {


    // LOCALIZA O PRODUTO PELO ID

    let produto = produtos.find(

        item => item.id === id

    );
```

```
if (produto == undefined) {
    return;
}

// GUARDA QUAL PRODUTO SERÁ EDITADO
idEdicao = id;

// PREENCHE O FORMULÁRIO
document.getElementById("nome").value =
    produto.nome;

document.getElementById("categoria").value =
    produto.categoria;

document.getElementById("preco").value =
    produto.preco;

document.getElementById("quantidade").value =
    produto.quantidade;
// ALTERA O TEXTO DO BOTÃO
document.getElementById("botaoSalvar").textContent =
    "💾 Salvar Alterações";
// LEVA A TELA ATÉ O CADASTRO
document.getElementById("cadastro").scrollIntoView({
    behavior: "smooth"
});
```

}

BLOCO 5 — Limpar o formulário

Comando: reset() limpa os campos do formulário.

COPIE E COLE ABAIXO

```
// LIMPA O FORMULÁRIO
function limparFormulario() {

    document.getElementById("formProduto").reset();

    // VOLTA AO MODO DE CADASTRO
    idEdicao = null;

    document.getElementById("botaoSalvar").textContent =
        "💾 Salvar Produto";

}
```

BLOCO 6 — Excluir produtos

Comandos: confirm() pede confirmação; DELETE solicita ao Python a exclusão do registro.

COPIE E COLE ABAIXO

```
// EXCLUI UM PRODUTO DO BANCO
async function excluirProduto(id) {

    let confirmar = confirm(
        "Deseja realmente excluir este produto?"
    );

    if (confirmar == false) {
```

```
        return;
    }

    try {

        // SOLICITA A EXCLUSÃO AO PYTHON
        let resposta = await fetch(
            "/api/produtos/" + id,
            { method: "DELETE" }
        );

        if (!resposta.ok) {
            throw new Error("Falha ao excluir");
        }

        alert("Produto excluído!");

        // ATUALIZA A TABELA
        await carregarProdutos();

    } catch (erro) {

        alert("Não foi possível excluir o produto.");

    }

}
```

BLOCO 7 — Carregamento inicial

Ao abrir o site, queremos que os produtos já apareçam automaticamente.

Comando: DOMContentLoaded executa a função quando a estrutura HTML termina de carregar.

COPIE E COLE ABAIXO

```
// CARREGA OS PRODUTOS QUANDO A PÁGINA ABRIR
document.addEventListener(
    "DOMContentLoaded",
    carregarProdutos
);
```

Pronto! O JavaScript está concluído.

7 — TESTE GERAL DO STOCKTECH

Agora vamos verificar se o front-end está realmente conectado ao banco de dados.

Atenção: antes de testar, confirme que todos os blocos do script.js foram copiados. O primeiro teste completo deve ser feito agora.

Missão 1 — Cadastrar

Cadastre os seguintes produtos:

Produto	**Preço**	**Quantidade**
Mouse Gamer	R$ 120,00	5
Teclado Mecânico	R$ 250,00	3
Headset	R$ 180,00	4
Webcam	R$ 200,00	2

Escolha as categorias correspondentes.

O sistema deverá apresentar:

4

Produtos

14

Unidades

R$ 2.470

Estoque

Missão 2 — Editar

Localize o Mouse Gamer e clique em Editar.

Altere a quantidade de 5 para 10 e salve.

O valor total do estoque deverá mudar automaticamente.

Missão 3 — Excluir

Exclua a Webcam.

Confirme a exclusão e verifique se o produto desapareceu da tabela.

Missão 4 — Testar o banco

Agora vem o teste mais importante!

1. Atualize a página.
2. Feche a aba do navegador.
3. Abra novamente http://127.0.0.1:8000.
4. Confira se os produtos continuam cadastrados.
5. Encerre o servidor com Ctrl + C no terminal.
6. Inicie novamente com python servidor.py ou py servidor.py.
7. Abra o sistema novamente.

Resultado esperado: os produtos continuam disponíveis, porque estão gravados no arquivo estoque.db.

8 — E SE APARECER UM ERRO?

Essa parte é para você consultar rapidamente durante a aula. 😂

Problema	**O que verificar**

python não reconhecido	Testar py servidor.py
Página não abre	Verificar se o servidor está rodando
Porta 8000 ocupada	Encerrar outro servidor que esteja usando essa porta
Botões não funcionam	Conferir script.js e os nomes das funções
Produto não aparece	Conferir se o servidor Python está funcionando
Erro IndentationError	Verificar os espaços no servidor.py
Arquivo .db não aparece	Conferir se servidor.py foi executado

Dois cuidados importantes para o Python:

No servidor.py, os métodos do_GET, do_POST, do_PUT e do_DELETE precisam ficar com a mesma indentação, dentro da classe Servidor. Já o último bloco, if __name__ == "__main__":, começa na margem esquerda.

Execute o servidor com o terminal aberto na pasta Aula13_StockTech. Não utilize o Live Server nesta aula, porque agora é o Python que entrega as páginas e atende à API.

9 — DESAFIO FINAL — EVOLUINDO O STOCKTECH

Agora é a vez de vocês trabalharem sem código pronto.

Missão: o cliente solicitou melhorias no sistema de estoque.

Escolha pelo menos duas melhorias:

- Alerta de estoque baixo: destacar produtos com menos de cinco unidades.
- Pesquisa: criar um campo para localizar produtos pelo nome.
- Nova categoria: acrescentar uma categoria e utilizá-la em um cadastro.
- Identidade visual: personalizar as cores, o cabeçalho e os botões.
- Indicador adicional: mostrar quantos produtos estão sem estoque.
- Ordenação: apresentar os produtos em ordem alfabética.

O aluno deverá testar e demonstrar as melhorias funcionando.

10 — FECHAMENTO DA AULA

Hoje construímos um sistema com:

HTML → estrutura

CSS → aparência e responsividade

JavaScript → interatividade e fetch()

Python → servidor e API

SQL → comandos de banco de dados

SQLite → armazenamento persistente

Também aplicamos as quatro operações do CRUD:

Operação	**SQL**	**Resultado**
Create	INSERT	Cadastrar
Read	SELECT	Consultar
Update	UPDATE	Editar
Delete	DELETE	Excluir

Diferença principal: no CinePlay, os registros eram armazenados no navegador. No StockTech, são armazenados em um arquivo de banco de dados acessado pelo servidor Python.

Esse banco continua local, no computador onde o servidor está rodando. Ainda não é um sistema publicado para vários usuários na internet.